EQUIPMENT LOANING MANAGEMENT SYSTEM

1. OBJECTIVES

The main objectives of this assignment are to:

- Create and start multiple threads to perform tasks concurrently (at the same time).
- Apply core threading concepts such as thread synchronization, shared resource management, and basic thread lifecycle control.

2. DESCRIPTION

You will build a multithreaded application that simulates a neighborhood equipment loan system — a shared **warehouse** where community members can borrow and return appliances and tools.

The application must handle concurrent operations realistically: multiple members may try to loan or return items at the same time, so your program must manage shared data safely across threads.

2.1 User Interface

You have **two options:** The user interface may be implemented as a text-based console application or, preferably, as a graphical user interface (GUI).

If you choose the GUI approach, you may either design your own or use the ready-made code provided in the Canvas module.

2.2 Core Data Model:

Entity (Class)	Fields	Description
Product	Id, name	An item available for loan (e.g., a drill, a ladder)
Member	Id, name	A neighborhood resident who can borrow products
LoanItem	Member, Product	Records that a specific member has borrowed a specific product

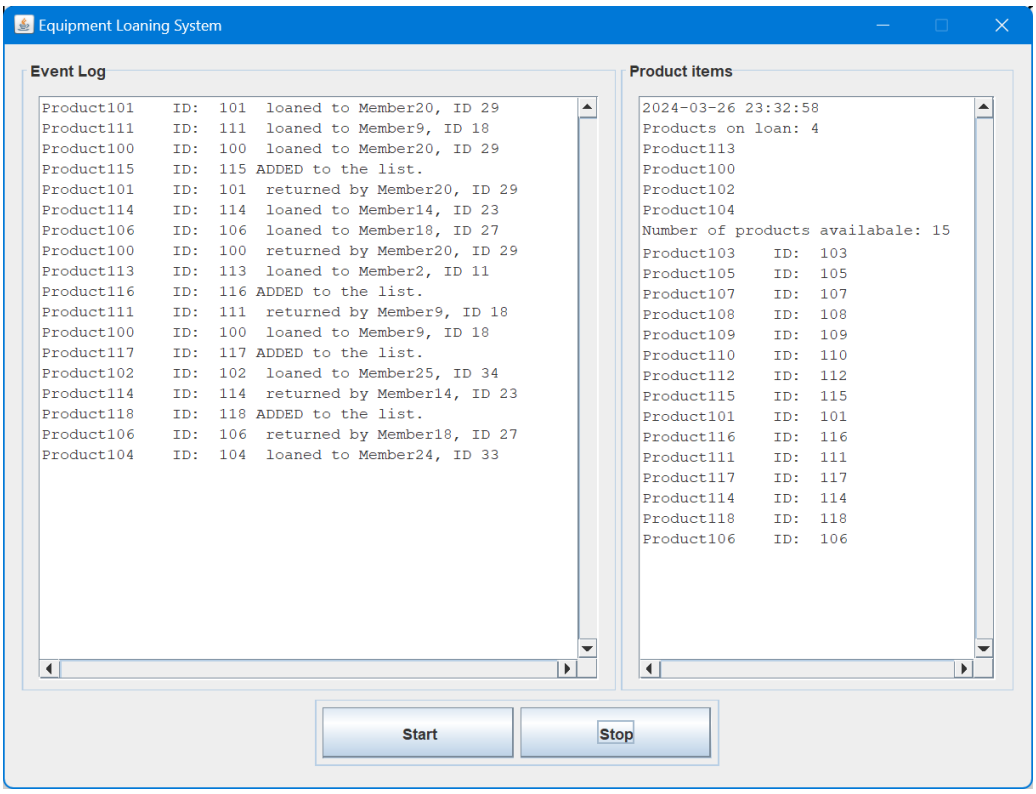
The above are at minimum and you may add more classes and attributes (fields) to the classes.

The system maintains three corresponding shared lists:

1. A product catalog — a list of all items (products) available in the warehouse-
2. A member registry — a list of all registered neighborhood members.
3. A loan registry (active loans) — a list of all products currently on loan.

These lists are shared between threads, meaning multiple threads may read and write them simultaneously. You must handle this carefully to avoid data corruption.

The screenshot below illustrates an example of the system running:



Hint:

Consider creating a dedicated coordinator class — such as **LoanSystemManager** — whose sole responsibility is to set up and manage the application at startup. This class would create the task objects, instantiate and start the threads, and coordinate access to the manager classes whenever list operations are needed. This keeps your main method clean and your application logic well-separated.

3. Simulation Notes

All system activity — loaning items, returning them, and any other operations — should be driven by **randomized timing** to realistically simulate real-world, concurrent behavior. In this simulation, **time is measured in seconds, where each second represents one day**.

To simulate the passage of time, use the **Thread.sleep()** method (or **Thread.Sleep()** in C#) with a randomly selected duration in seconds. For example, to simulate a member holding a loaned item for ***n*** days, the responsible thread should sleep for ***n*** seconds:

```
Thread.sleep(sleepTimeInSeconds); // Java
Thread.Sleep(sleepTimeInSeconds); // C#
```

This approach applies to all timed actions in the simulation — not just loans. For instance:

- A **loan** lasts *n* days → the thread sleeps for *n* seconds before triggering a return.
- A **new item** might arrive after a delay → the thread sleeps before adding it to the inventory.
- A **member** might wait a random number of days before requesting a loan.

Important: The use of a **Timer** control is **not permitted**. All timing must be handled exclusively through Thread.sleep() / Thread.Sleep()!

4. Manager Classes:

It is recommended that each of the three core lists is managed by a **dedicated manager class**: **ProductManager**, **MemberManager**, and **LoanItemManager**. Each manager is a class that internally maintains a list of its respective objects — for example, ProductManager holds a list of Product objects, MemberManager holds a list of Member objects, and so on.

For storing the items, you may use a plain array, but it is strongly recommended to use a built-in collection type provided by your chosen language, as these offer more flexibility and built-in functionality:

- **Java:** ArrayList, LinkedList, or other classes from the java.util package
- **C#:** List<T>, LinkedList<T>, or other types from the System.Collections.Generic namespace

Each manager class should include methods for interacting with its internal list, including at minimum:

- **Add** — insert a new object into the list
- **Remove** — delete an existing object from the list
- **Get** — retrieve a specific object or the entire list.
- Additional methods as needed, such as checking if the list is empty or selecting a random item.

Advanced hint: You may consider creating a generic manager class that maintains a List<T> and includes all common methods (Add, Remove, Change, Get, etc.) and then utilize this class within the manager classes (using for example inheritance).

5. Tasks and Threads

The system's activity is divided into **four concurrent tasks**, each running on its own dedicated thread. Each task is implemented as a class with a Run() method (or run() in Java) that contains the task's logic inside a loop and represents the entry point of each task's thread logic.

Note: How this method is implemented depends on your chosen language — refer to the reading material for the thread creation approach applicable to Java or C#.

5.1 Thread 1 — LoanTask

This thread continuously simulates the loaning of items. At **random intervals** (e.g., every 2–6 seconds), it performs the following steps:

1. Randomly selects an available **product** from the product list.
2. Randomly selects a **member** from the member list.
3. Creates a **LoanItem** object linking the two and adds it to the loan list.
4. Removes the selected product from the product list — ensuring the product list only contains items **currently available** for loaning.

   As a simplification, when a returned product is added back, it may be appended to the end of the list, not in its original position.

5. The member is **not** removed from the member list, allowing the same member to borrow additional items later. If the product list is **empty** (meaning all items are currently on loan), this thread should pause and wait until at least one product becomes available again before proceeding.

5.2 Thread 2 — ReturnTask

This thread simulates members returning borrowed items. After a **random delay** (e.g., 3–15 seconds, representing the loan period in days), it performs the following steps:

1. Randomly selects a **LoanItem** from the loan list.
2. Adds the associated **product** back to the product list, making it available for loaning again.
3. Removes the **LoanItem** from the loan list.

The member is **not** re-added to the member list since they were never removed — the member list remains constant throughout the application's runtime.

5.3 Thread 3 — AdminTask

This thread simulates administrative warehouse activity. At **random intervals** (e.g., every 6–16 seconds), it generates and adds a new test product to the product list, simulating new stock arriving at the warehouse.

Optional Extension: To make this task more realistic, the thread could also simulate items being taken out of service due to loss or damage. In this case, the thread randomly chooses between **adding** a new product or **removing** an existing one from the product list, with additions occurring more frequently than removals.

5.4 Thread 4 — UpdateGUITask

This thread is solely responsible for keeping the user interface up to date. At regular intervals (e.g., every 1–2 seconds), it refreshes the display to show:

- The current list of **products** available for loan
- The current list of **active loans** (**LoanItems**)

This thread does **not** modify any data — it only reads and displays the current state of the system.

Task Summary

Thread	Example Interval (seconds)	Responsibility
LoanTask	2–6	Loans a random product to a random member
ReturnTask	3–15	Returns a randomly selected loaned item
AdminTask	6–16	Adds (and optionally removes) products
UpdateGUITask	1–2	Refreshes the UI with current system state

5.5 1.5 Threads

Each of the four tasks described above must be executed by a dedicated thread, running concurrently with the others. To set this up:

Create an instance of each task class (LoanTask, ReturnTask, AdminTask, UpdateGUITask).

1. Create a thread for each task instance, following the thread creation approach for your chosen language as covered in your reading material.
2. Start all four threads together, only after all have been created. Starting them as a group, rather than starting each thread immediately after creating it, maximizes the overlap in their execution, giving a more observable simulation of concurrent behavior.

   Once all four threads are started, they will run **simultaneously**, each independently executing its assigned task.

Each thread executes its task logic inside a loop controlled by a **Boolean** flag (**isRunning**). Regardless of language, the core structure follows this pattern:

```
// Java and C# — same syntax
while (isRunning)
{
    // Task logic goes here
}
```

To start a thread, set isRunning = true before starting it. To stop it gracefully, set isRunning = false; the thread will finish its current iteration and then exit the loop.

Important: Do not use any deprecated Thread methods to stop or suspend a thread (such as **Thread.stop()** or **Thread.suspend()** in Java, or their equivalents in C#). Always use the **isRunning** flag approach for safe and clean thread termination.

6. Test Values

When the application starts, it should automatically populate the system with an initial set of test data:

- **Products** — generate at least 15 products (e.g., *Product100*, *Product101*, *Product102*, ..)
- **Members** — generate at least 25 members (e.g., *Member1*, *Member2*, *Member3*, ...)

You are encouraged to use more descriptive and realistic names rather than generic placeholders. For example:

- Products: "Makita Lawn Mower", "Black & Decker Drill", "Bosch Pressure Washer"
- Members: *"House 32"*, *"Mr. Anderson"*, *"Mrs. Patel"*

As a cleaner alternative, consider preparing your product and member data in **external text files** and loading them into the application at startup. This makes it easy to adjust test data without modifying or recompiling your code.

The responsibility for generating and loading test data should be placed within the **ProductManager** and **MemberManager** classes respectively, keeping initialization logic organized and clearly separated from the rest of the application.

Hint: Consider creating a dedicated coordinator class — such as **LoanSystemManager** — whose sole responsibility is to set up and manage the application at startup. This class would create the task objects, instantiate and start the threads, and coordinate access to the manager classes whenever list operations are needed. This keeps your main method clean and your application logic well-separated.

7. Submission

Review the code quality, assessment, and submission guidelines (“Kodkvalitet och inlämning av uppgifter”) on the start page on Canvas to ensure you meet all general requirements.

Compress your entire project—including all files, folders, and subfolders—into a single .zip, .rar, or .7z archive. Upload the compressed file to the Assignment page on Canvas, preferably before your presentation.

Good Luck!

Farid Naisan,
Instructor